

循序渐进，学习开发 xv6-riscv

第 7 章 陷阱 (Trap)

汪辰



目录

01

控制流与陷阱 (Trap)

02

Trap 相关 CSR

03

Trap 处理流程



01

控制流与陷阱 (Trap)



一些基本概念

- **控制流 (Control Flow)**

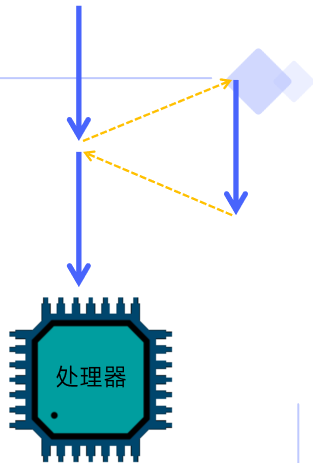
- ✓ branch, jump

- 异常控制流 (Exceptional Control Flow, 简称 ECP)

- ✓ **Exception (异常)**

- ✓ **Interrupt (中断)**

- RISC-V 把 ECP 统称为 **陷阱 (Trap)**



A decorative graphic on the left side of the slide. It features two concentric circles with a light blue glow. A thin grey arc starts from the top of the outer circle and curves around to the bottom. In the top right corner, there is a solid blue rectangle.

02

Trap 相关 CSR

M模式下 Trap 代理相关 CSR 一览

Table 7. Currently allocated RISC-V machine-level CSR addresses.

⋮	⋮	⋮	⋮
Machine Trap Setup			
⋮	⋮	⋮	⋮
0x302	MRW	medeleg	Machine exception delegation register.
0x303	MRW	mideleg	Machine interrupt delegation register.
⋮	⋮	⋮	⋮

M模式下 Trap 代理相关 CSR 一览

- 默认情况下，所有特权级别的 Trap 均在 Machine 模式下处理。为了提高性能，可以将某些异常和中断交由较低特权级别直接处理。medeleg 和 mideleg 即用于设置委托关系的寄存器，
- 在实现了 S 模式的 harts 系统中，medeleg 和 mideleg 寄存器必须存在。
- 在没有实现 S 模式的 harts 系统中，medeleg 和 mideleg 寄存器不应存在。

寄存器	用途说明
medeleg (Machine Exception Delegation)	将特定的异常处理委托给 S-mode，避免每次异常都陷入 M-mode
mideleg (Machine Interrupt Delegation)	将特定的中断处理委托给 S-mode，使 S-mode 能直接处理中断而不经 M-mode。

medeleg (Machine Exception Delegation)

63

0

Synchronous Exceptions (WARL)

64

Figure 11. Machine Exception Delegation (medeleg) register.

- 将特定的异常处理委托给 S-mode，避免每次异常都陷入 M-mode。
- medeleg 为 M 模式下可以处理的每个同步异常分配了一个位位置，该位位置的索引等于 mcause 寄存器中返回的值（见右表中的 Exception Code 列）。

Table 14. Machine cause (mcause) register values after trap.

Interrupt	Exception Code	Description
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10	Reserved
0	11	Environment call from M-mode
0	12	Instruction page fault
0	13	Load page fault
0	14	Reserved
0	15	Store/AMO page fault
0	16	Double trap
0	17	Reserved
0	18	Software check
0	19	Hardware error
0	20-23	Reserved
0	24-31	Designated for custom use
0	32-47	Reserved
0	48-63	Designated for custom use
0	≥64	Reserved

mideleg (Machine Interrupt Delegation)

MXLEN-1

0

Interrupts (WARL)

MXLEN

Figure 12. Machine Interrupt Delegation (mideleg) Register.

- 将特定的中断处理委托给 S-mode，使 S-mode 能直接处理中断而不经 M-mode。
- mideleg 寄存器保存各个中断的委托位，其位布局与 mip 寄存器中的位布局相匹配。（见右表中的 Exception Code 列）。

Table 14. Machine cause (mcause) register values after trap.

Interrupt	Exception Code	Description
1	0	Reserved
1	1	Supervisor software interrupt
1	2	Reserved
1	3	Machine software interrupt
1	4	Reserved
1	5	Supervisor timer interrupt
1	6	Reserved
1	7	Machine timer interrupt
1	8	Reserved
1	9	Supervisor external interrupt
1	10	Reserved
1	11	Machine external interrupt
1	12	Reserved
1	13	Counter-overflow interrupt
1	14-15	Reserved
1	≥16	Designated for platform use

S模式下 Trap 相关 CSR 一览

Table 5. Currently allocated RISC-V supervisor-level CSR addresses.

Number	Privilege	Name	Description
Supervisor Trap Setup			
0x100	SRW	sstatus	Supervisor status register.
0x104	SRW	sie	Supervisor interrupt-enable register.
0x105	SRW	stvec	Supervisor trap handler base address.
0x106	SRW	scounteren	Supervisor counter enable.
⋮	⋮	⋮	⋮
Supervisor Trap Handling			
0x140	SRW	sscratch	Supervisor scratch register.
0x141	SRW	sepc	Supervisor exception program counter.
0x142	SRW	scause	Supervisor trap cause.
0x143	SRW	stval	Supervisor trap value.
0x144	SRW	sip	Supervisor interrupt pending.
0xDA0	SRO	scountovf	Supervisor count overflow.

S模式下 Trap 相关 CSR 一览

寄存器	用途说明
stvec (Supervisor Trap Vector Base Address)	它保存异常发生时和处理器跳转地址相关的信息。
sepc (Supervisor Exception Program Counter)	当 trap 发生时, hart 会将发生 trap 所对应的指令的地址值 (pc) 保存在 sepc 中。
scause (Supervisor Cause)	当 trap 发生时, hart 会设置该寄存器通知我们 trap 发生的原因。
sip (Supervisor Interrupt Pending)	它列出目前已发生等待处理的中断。
sie (Supervisor Interrupt Enable)	用于进一步控制 (打开和关闭) software interrupt/timer interrupt/external interrupt
stval (Supervisor Trap Value)	它保存了 exception 发生时的附加信息: 譬如访问地址出错时的地址信息、或者执行非法指令时的指令本身, 对于其他异常, 它的值为 0。
sstatus (Supervisor Status)	用于跟踪和控制 hart 的当前操作状态 (特别地, 包括关闭和打开全局中断)。

stvec (Supervisor Trap Vector Base Address)

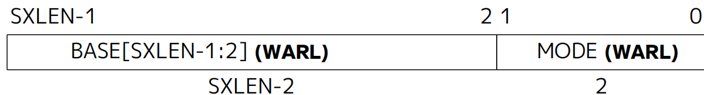


Figure 51. Supervisor trap vector base address (stvec) register.

- BASE: trap 入口函数的基地址的高 62 位 (RV64), , 必须保证四字节对齐。
- MODE: 进一步用于控制入口函数的地址配置方式:
 - ✓ Direct: 所有的 exception 和 interrupt 发生后 PC 都跳转到 BASE 指定的地址处。
 - ✓ Vectored: exception 处理方式同 Direct; 但 interrupt 的入口地址以数组方式排列。

Table 31. Encoding of stvec MODE field.

Value	Name	Description
0	Direct	All exceptions set pc to BASE.
1	Vectored	Asynchronous interrupts set pc to BASE+4×cause.
≥2		Reserved

stvec (Supervisor Trap Vector Base Address)

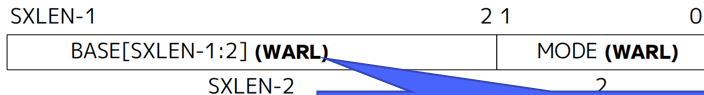


Figure 51. Supervisor trap vector

Write Any Values, Reads Legal Values (WARL)

- BASE: trap 入口函数的基地址的高 62 位 (RV64), , 必须保证四字节对齐。
- MODE: 进一步用于控制入口函数的地址配置方式:
 - ✓ Direct: 所有的 exception 和 interrupt 发生后 PC 都跳转到 BASE 指定的地址处。
 - ✓ Vectored: exception 处理方式同 Direct; 但 interrupt 的入口地址以数组方式排列。

Table 31. Encoding of stvec MODE field.

Value	Name	Description
0	Direct	All exceptions set pc to BASE.
1	Vectored	Asynchronous interrupts set pc to BASE+4×cause.
≥2		Reserved

stvec (Supervisor Trap Vector Base Address)

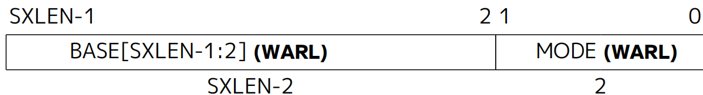


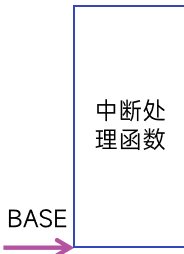
Figure 51. Supervisor trap vector base address (stvec) register.

- BASE: trap 入口函数的基地址的高 62 位 (RV64), 必须保证四字节对齐。
- MODE: 进一步用于控制入口函数的地址配置方式:
 - ✓ Direct: 所有的 exception 和 interrupt 发生后 PC 都跳转到 BASE 指定的地址处。
 - ✓ Vectored: exception 处理方式同 Direct; 但 interrupt 的入口地址以数组方式排列。

Table 31. Encoding of stvec MODE field.

Value	Name	Description
0	Direct	All exceptions set pc to BASE.
1	Vectored	Asynchronous interrupts set pc to BASE+4×cause.
≥2		Reserved

MODE = Direct



低地址

stvec (Supervisor Trap Vector Base Address)

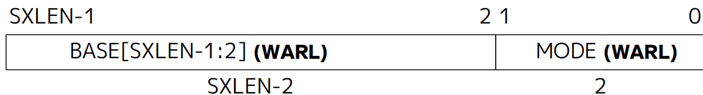


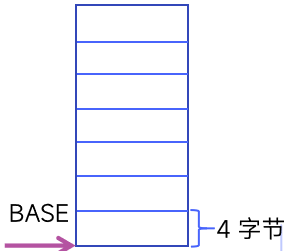
Figure 51. Supervisor trap vector base address (stvec) register.

- BASE: trap 入口函数的基地址的高 62 位 (RV64), , 必须保证四字节对齐。
- MODE: 进一步用于控制入口函数的地址配置方式:
 - ✓ Direct: 所有的 exception 和 interrupt 发生后 PC 都跳转到 BASE 指定的地址处。
 - ✓ Vectored: exception 处理方式同 Direct; 但 interrupt 的入口地址以数组方式排列。

Table 31. Encoding of stvec MODE field.

Value	Name	Description
0	Direct	All exceptions set pc to BASE.
1	Vectored	Asynchronous interrupts set pc to BASE+4×cause.
≥2		Reserved

MODE = Vectored



低地址

sepc (Supervisor Exception Program Counter)

SXLEN-1

0

sepc

SXLEN

Figure 58. Supervisor exception program counter register.

- 在 S 模式下当 trap 发生时，pc 会被替换为 stvec 设定的地址，同时 hart 会设置 sepc 为当前指令或者下一条指令的地址，当我们需要退出 trap 时可以调用特殊的 sret 指令，该指令会将 sepc 中的值恢复到 pc 中（实现返回的效果）。
- 在处理 trap 的程序中我们可以修改 sepc 的值达到改变 sret 返回地址的目的。

scause (Supervisor Cause)



Figure 59. Supervisor Cause (scause) register.

- 在 S 模式下当 trap 发生时，hart 会设置该寄存器通知我们 trap 发生的原因。
- 最高位 Interrupt 为 1 时标识了当前 trap 为 interrupt，否则是 exception。
- 剩余的 Exception Code 用于标识具体的 interrupt 或者 exception 的种类。

scause (Supervisor Cause)

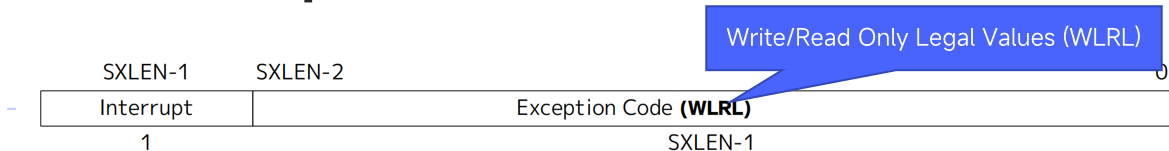


Figure 59. Supervisor Cause (scause) register.

- 在 S 模式下当 trap 发生时，hart 会设置该寄存器通知我们 trap 发生的原因。
- 最高位 Interrupt 为 1 时标识了当前 trap 为 interrupt，否则是 exception。
- 剩余的 Exception Code 用于标识具体的 interrupt 或者 exception 的种类。

scause (Supervisor Cause)

Table 32. Supervisor cause (scause) register values after trap.

Interrupt	Exception Code	Description
	0	Reserved
1	1	Supervisor software interrupt
1	2-4	Reserved
1	5	Supervisor timer interrupt
1	6-8	Reserved
1	9	Supervisor external interrupt
1	10-12	Reserved
1	13	Counter-overflow interrupt
1	14-15	Reserved
1	≥16	Designated for platform use
	0	Instruction address misaligned
	0	1 Instruction access fault
	0	2 Illegal instruction
	0	3 Breakpoint
	0	4 Load address misaligned
	0	5 Load access fault
	0	6 Store/AMO address misaligned
	0	7 Store/AMO access fault
	0	8 Environment call from U-mode
	0	9 Environment call from S-mode
	0	10-11 Reserved
	0	12 Instruction page fault
	0	13 Load page fault
	0	14 Reserved
	0	15 Store/AMO page fault
	0	16-17 Reserved
	0	18 Software check
	0	19 Hardware error
	0	20-23 Reserved
	0	24-31 Designated for custom use
	0	32-47 Reserved
	0	48-63 Designated for custom use
	0	≥64 Reserved

中断

异常

Table 15. Synchronous exception priority in decreasing priority order.

Priority	Exc.Code	Description
Highest	3	Instruction address breakpoint
		During instruction address translation:
	12, 1	First encountered page fault or access fault
		With physical address for instruction:
	1	Instruction access fault
		2 Illegal instruction
	0	Instruction address misaligned
	8, 9, 11	Environment call
	3	Environment break
	3	Load/store/AMO address breakpoint
		Optionally:
	4, 6	Load/store/AMO address misaligned
		During address translation for an explicit memory access:
	13, 15, 5, 7	First encountered page fault or access fault
		With physical address for an explicit memory access:
	5, 7	Load/store/AMO access fault
		If not higher priority:
Lowest	4, 6	Load/store/AMO address misaligned

stval (Supervisor Trap Value)

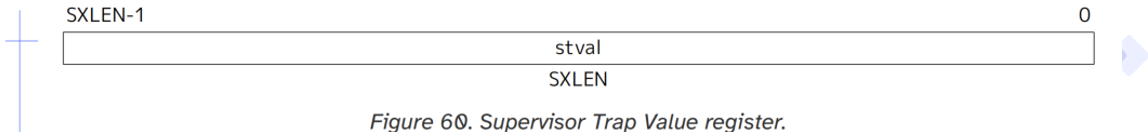


Figure 60. Supervisor Trap Value register.

- 在 S 模式下当 trap 发生时，除了通过 scause 可以获取 exception 的种类 code 值外，hart 还提供了 stval 来提供 exception 的其他信息来辅助我们执行更进一步的操作。
- 具体的辅助信息由特定的硬件实现定义，RISC-V 规范没有定义具体的值。但规范定义了一些行为，譬如辅助信息可以是访问地址出错时的地址信息、或者执行非法指令时的指令本身等，具体阅读 RISC-V 规范。

sstatus (Supervisor Status)

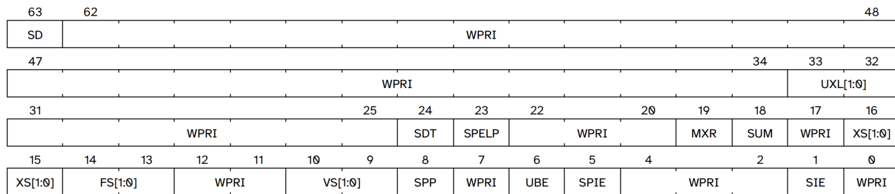


Figure 50. Supervisor-mode status (sstatus) register when SXLEN=64.

- **SIE** (Supervisor Interrupt Enable)：用于打开（1）或者关闭（0）S 模式下的 **全局** 中断。在 U 模式下忽略 SIE 的值并缺省 **全局** 中断总是打开的。在 S 模式下当 trap 发生时，hart 会自动将 SIE 设置为 0。
- **SPIE** (Supervisor Previous Interrupt Enable)：在 S 模式下当 trap 发生时用于保存 trap 发生之前的 SIE 的值。sret 时将 SIE 恢复为 SPIE 的值，同时将 SPIE 设置为 1。
- **SPP** (Supervisor Previous Privilege level)：在 S 模式下当 trap 发生时用于保存 trap 发生之前的权限级别值（0/1），sret 时根据 SPP 恢复权限级别。
- 其他标志位涉及内存访问权限、虚拟内存控制等，暂不考虑。

Table 1. RISC-V privilege levels.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	
3	11	Machine	M

sstatus (Supervisor Status)

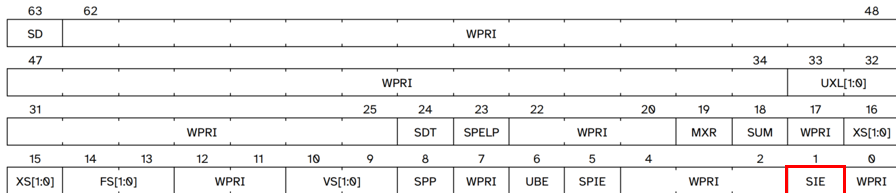


Figure 50. Supervisor-mode status (sstatus) register when SXLEN=64.

- SIE** (Supervisor Interrupt Enable)：用于打开（1）或者关闭（0）S 模式下的 **全局** 中断。在 U 模式下忽略 SIE 的值并缺省 **全局** 中断总是打开的。在 S 模式下当 trap 发生时，hart 会自动将 SIE 设置为 0。
- SPIE** (Supervisor Previous Interrupt Enable)：在 S 模式下当 trap 发生时用于保存 trap 发生之前的 SIE 的值。sret 时将 SIE 恢复为 SPIE 的值，同时将 SPIE 设置为 1。
- SPP** (Supervisor Previous Privilege level)：在 S 模式下当 trap 发生时用于保存 trap 发生之前的权限级别值（0/1），sret 时根据 SPP 恢复权限级别。
- 其他标志位涉及内存访问权限、虚拟内存控制等，暂不考虑。

Table 1. RISC-V privilege levels.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	
3	11	Machine	M

sstatus (Supervisor Status)

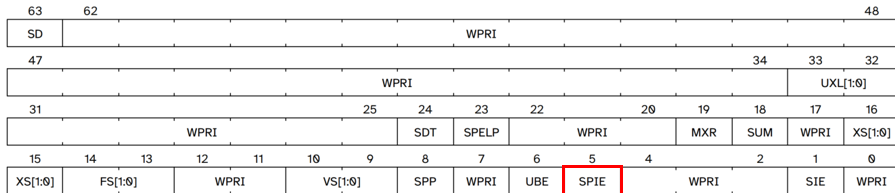


Figure 50. Supervisor-mode status (sstatus) register when SXLEN=64.

- SIE** (Supervisor Interrupt Enable)：用于打开（1）或者关闭（0）S 模式下的 **全局** 中断。在 U 模式下忽略 SIE 的值并缺省 **全局** 中断总是打开的。在 S 模式下当 trap 发生时，hart 会自动将 SIE 设置为 0。
- SPIE** (Supervisor Previous Interrupt Enable)：在 S 模式下当 trap 发生时用于保存 trap 发生之前的 SIE 的值。sret 时将 SIE 恢复为 SPIE 的值，同时将 SPIE 设置为 1。
- SPP** (Supervisor Previous Privilege level)：在 S 模式下当 trap 发生时用于保存 trap 发生之前的权限级别值（0/1），sret 时根据 SPP 恢复权限级别。
- 其他标志位涉及内存访问权限、虚拟内存控制等，暂不考虑。

Table 1. RISC-V privilege levels.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	
3	11	Machine	M

sstatus (Supervisor Status)

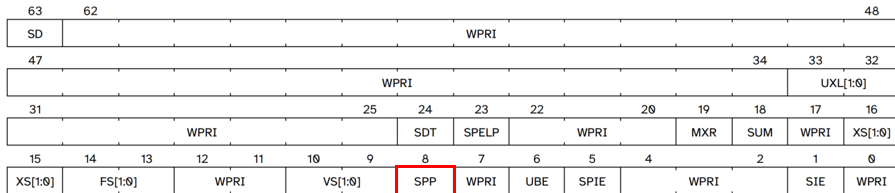


Figure 50. Supervisor-mode status (sstatus) register when SXLEN=64.

- **SIE** (Supervisor Interrupt Enable)：用于打开（1）或者关闭（0）S 模式下的 **全局** 中断。在 U 模式下忽略 SIE 的值并缺省 **全局** 中断总是打开的。在 S 模式下当 trap 发生时，hart 会自动将 SIE 设置为 0。
- **SPIE** (Supervisor Previous Interrupt Enable)：在 S 模式下当 trap 发生时用于保存 trap 发生之前的 SIE 的值。sret 时将 SIE 恢复为 SPIE 的值，同时将 SPIE 设置为 1。
- **SPP** (Supervisor Previous Privilege level)：在 S 模式下当 trap 发生时用于保存 trap 发生之前的权限级别值（0/1），sret 时根据 SPP 恢复权限级别。
- 其他标志位涉及内存访问权限、虚拟内存控制等，暂不考虑。

Table 1. RISC-V privilege levels.

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	Reserved	
3	11	Machine	M

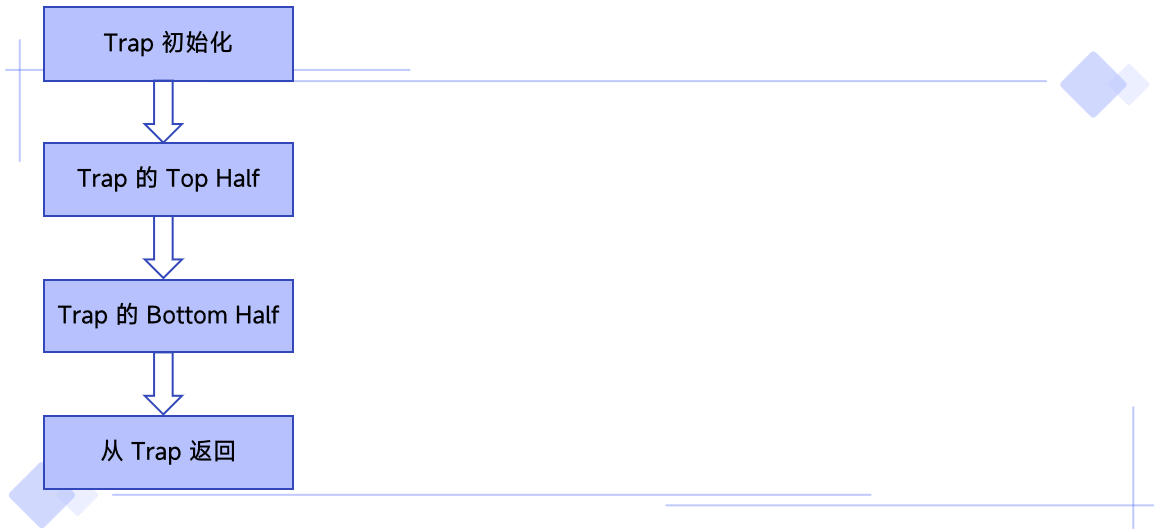


03

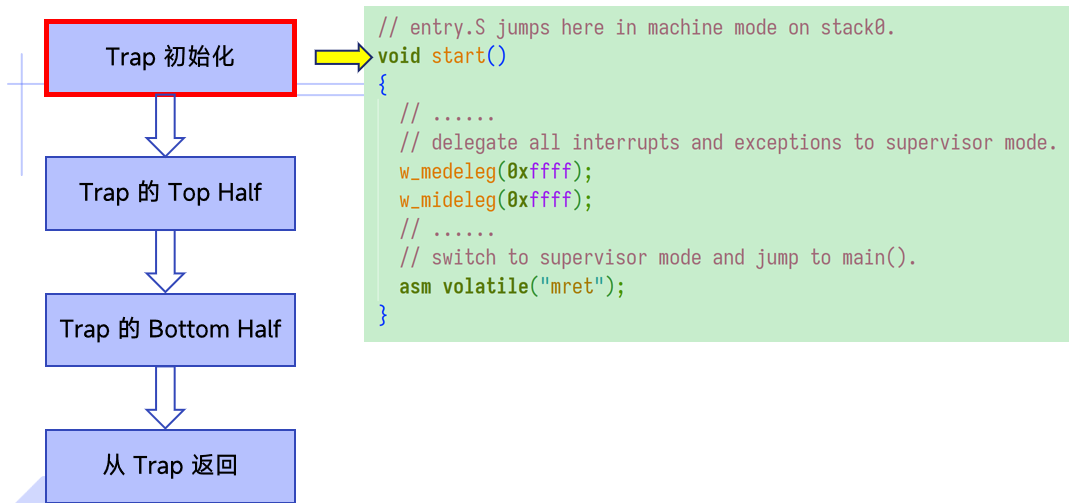
Trap 处理流程



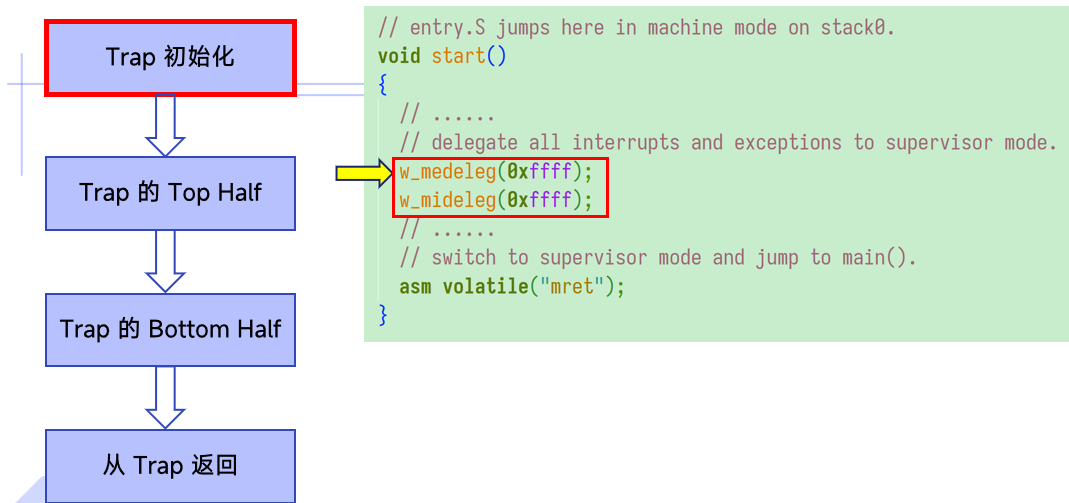
Trap 处理流程(S模式下)



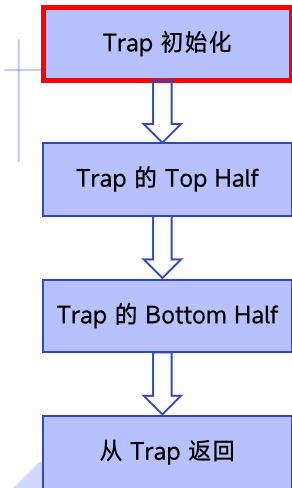
Trap 处理流程(S模式下)



Trap 处理流程(S模式下)

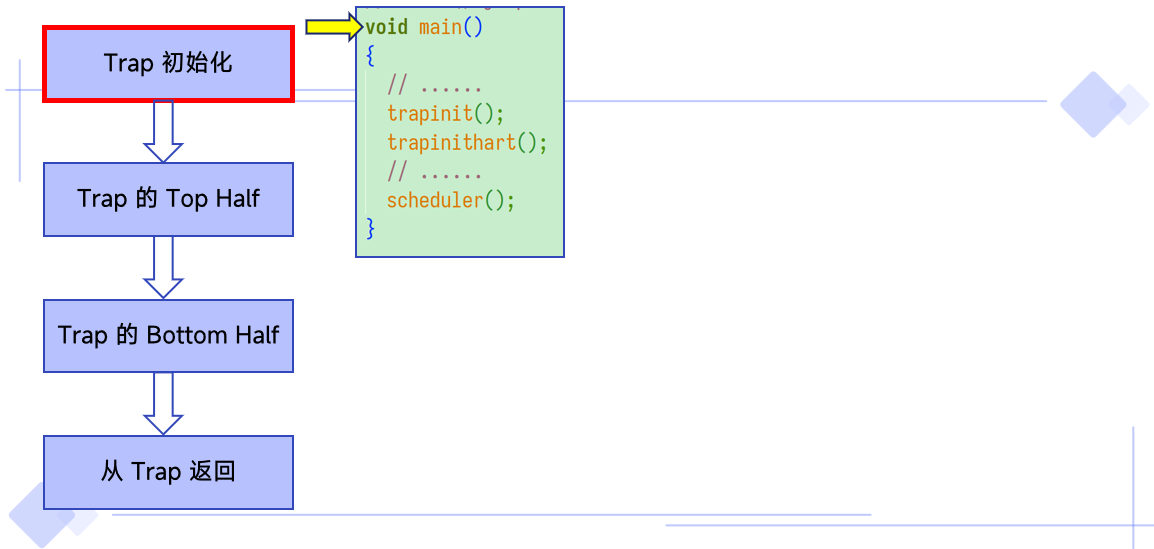


Trap 处理流程(S模式下)

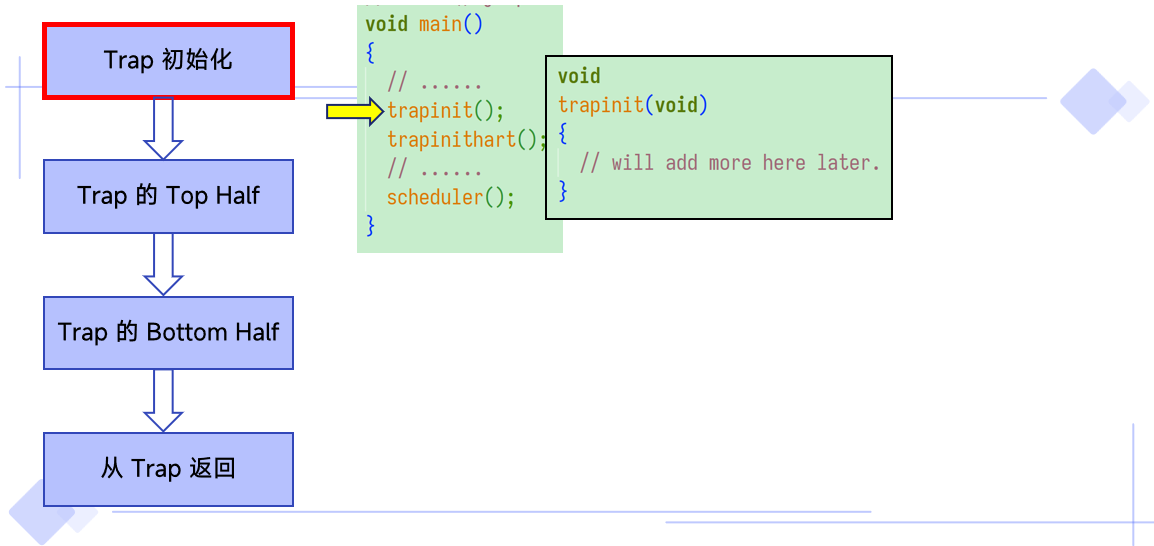


```
// entry.S jumps here in machine mode on stack0.  
void start()  
{  
    // .....  
    // delegate all interrupts and exceptions to supervisor mode.  
    w_medeleg(0xffff);  
    w_mideleg(0xffff);  
    // .....  
    // switch to supervisor mode and jump to main().  
    asm volatile("mret");  
}
```

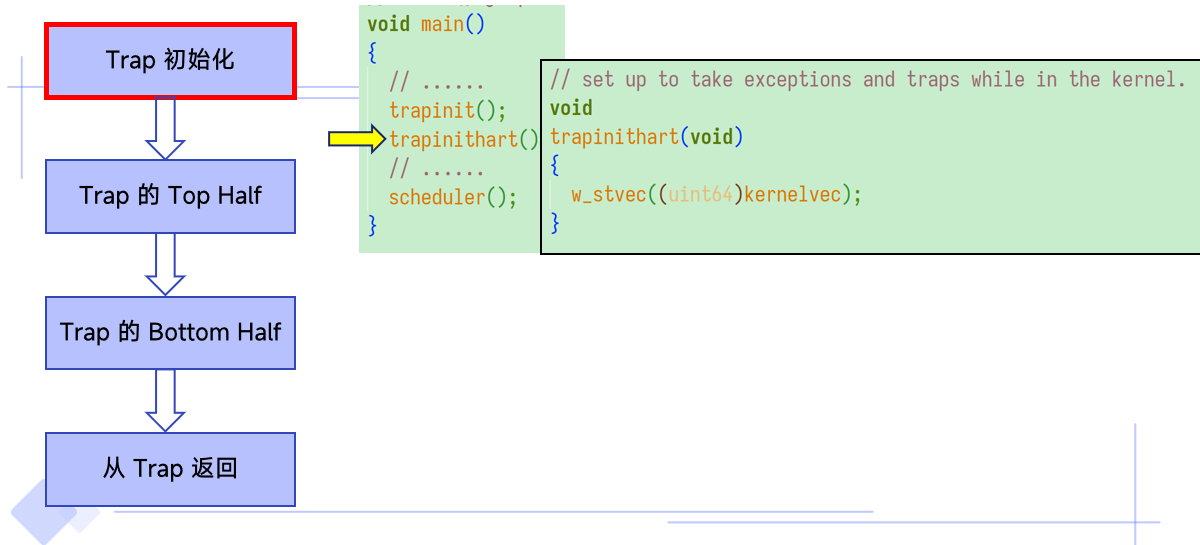
Trap 处理流程(S模式下)



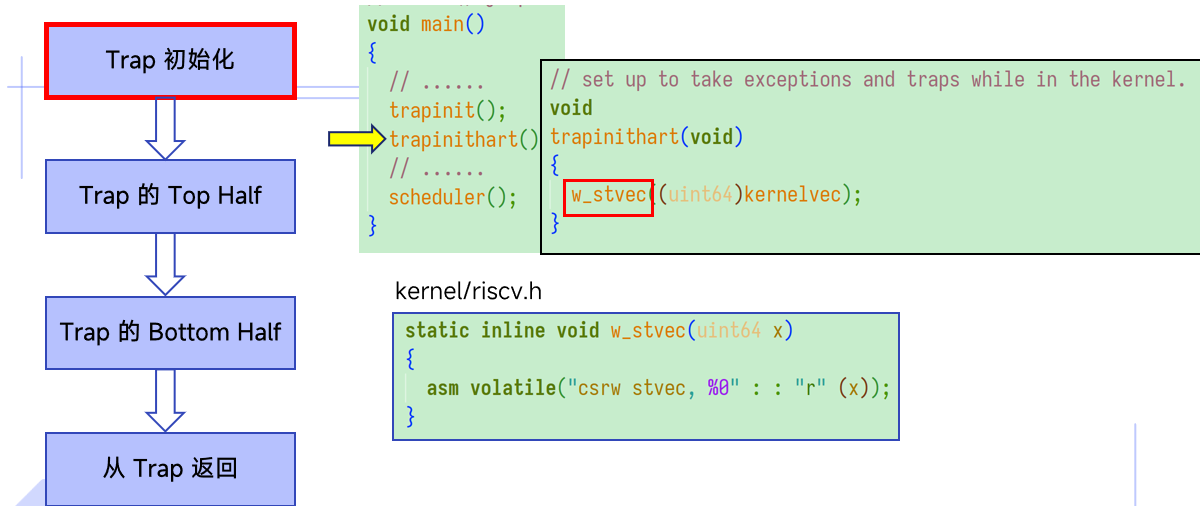
Trap 处理流程(S模式下)



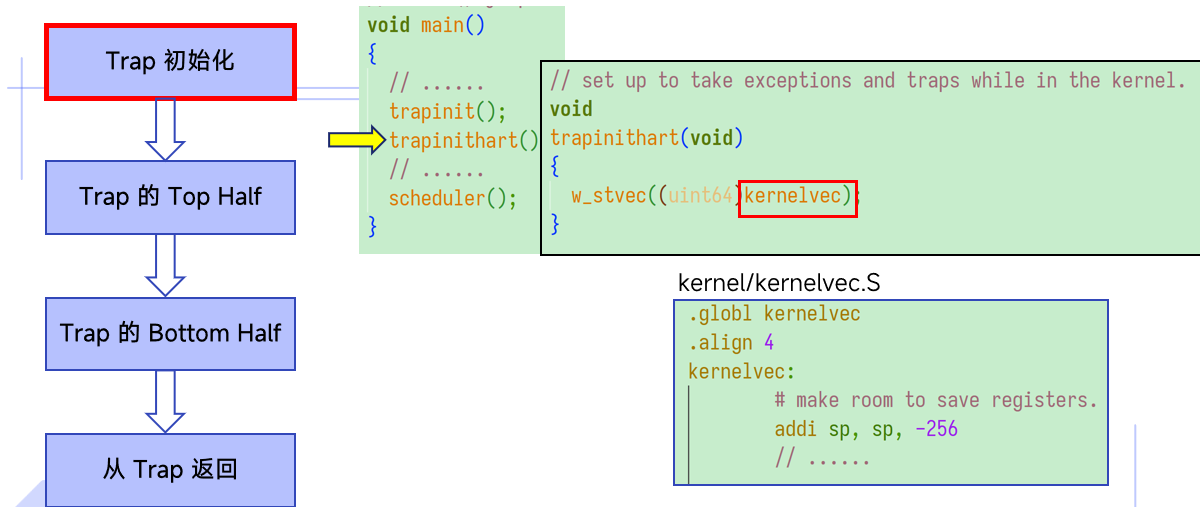
Trap处理流程(S模式下)



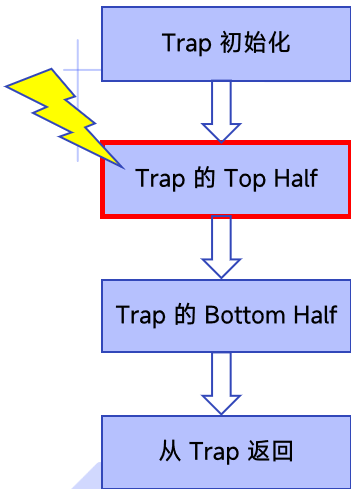
Trap处理流程(S模式下)



Trap处理流程(S模式下)

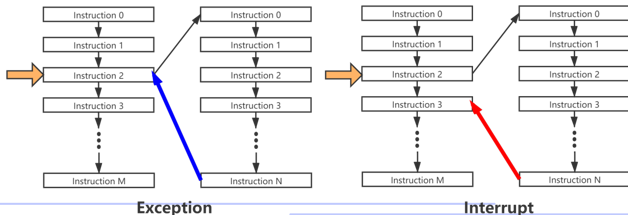


Trap 处理流程(S模式下)



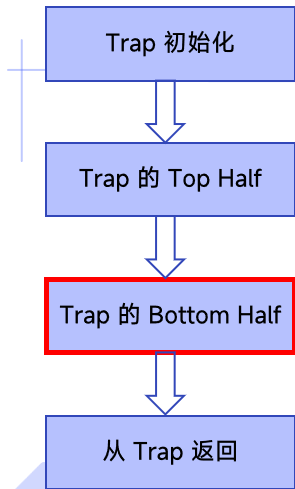
Trap 发生, Hart 自动执行如下状态转换

- 把 sstatus 的 SIE 值复制到 SPIE 中, 清除 sstatus 中的 SIE 标志位, 效果是 S 模式下的全局中断被禁止。
- 根据 trap 的种类设置 scause, 并根据需要为 stval 设置附加信息。
- 将 trap 发生之前的权限模式保存在 sstatus 的 SPP 域中, 再把 hart 权限模式更改为 Supervisor 模式 (前提是我们已经通过 medeleg 和 mideleg 实现了委托, 否则默认切换到 Machine 模式)。
- 设置 sepc。需要注意的是, 对于 exception, sepc 指向导致异常的指令; 对于 interrupt, 它指向被中断的指令的下一条指令的位置。



- PC 被设置为 stvec。

Trap处理流程(S模式下)



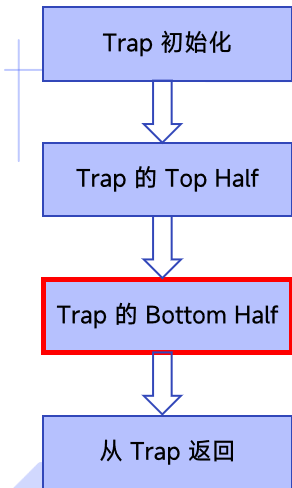
```
.globl kerneltrap
.globl kernelvec
.align 4
kernelvec:
    # make room to save registers.
    addi sp, sp, -256
    # save caller-saved registers.
    sd ra, 0(sp)
    # .....
    sd t6, 240(sp)

    # call the C trap handler in trap.c
    call kerneltrap

    # restore registers.
    ld ra, 0(sp)
    # .....
    ld t6, 240(sp)
    addi sp, sp, 256

    # return to whatever we were doing in the kernel.
    sret
```

Trap 处理流程 (S 模式下)



```
.globl kerneltrap
.globl kernelvec
.align 4
kernelvec:
```

```
# make room to save registers.
addi sp, sp, -256
# save caller-saved registers.
sd ra, 0(sp)
# .....
sd t6, 240(sp)
```

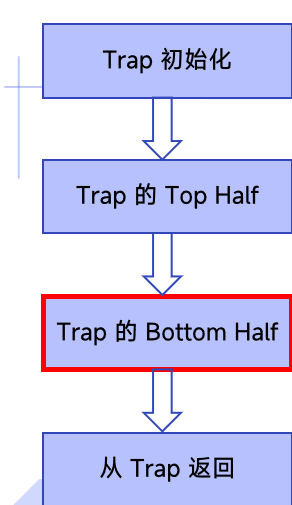
保存 (save) 当前控制流的上下文信息, 包括 ra, gp, tp, t0~t6, a0~a7 这些 caller-saved 寄存器

```
# call the C trap handler in trap.c
call kerneltrap
```

```
# restore registers.
ld ra, 0(sp)
# .....
ld t6, 240(sp)
addi sp, sp, 256
```

```
# return to whatever we were doing in the kernel.
sret
```

Trap 处理流程(S模式下)



```
.globl kerneltrap
.globl kernelvec
.align 4
kernelvec:
```

```
# make room to save registers.
addi sp, sp, -256
# save caller-saved registers.
sd ra, 0(sp)
# .....
sd t6, 240(sp)
```

保存 (save) 当前控制流的上下文信息, 包括 ra, gp, tp, t0~t6, a0~a7 这些 caller-saved 寄存器

```
# call the C
call kerneltrap

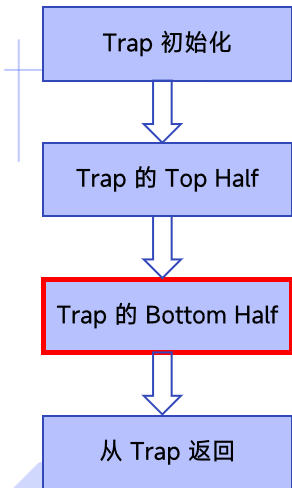
# restore registers
ld ra, 0(sp)
# .....
ld t6, 240(sp)
addi sp, sp, 256

# return to user
sret
```

Table 1. Integer register convention

Name	ABI Mnemonic	Meaning	Preserved across calls?
x0	zero	Zero	— (Immutable)
x1	ra	Return address	No
x2	sp	Stack pointer	Yes
x3	gp	Global pointer	— (Unallocatable)
x4	tp	Thread pointer	— (Unallocatable)
x5 - x7	t0 - t2	Temporary registers	No
x8 - x9	s0 - s1	Callee-saved registers	Yes
x10 - x17	a0 - a7	Argument registers	No
x18 - x27	s2 - s11	Callee-saved registers	Yes
x28 - x31	t3 - t6	Temporary registers	No

Trap 处理流程(S模式下)



```
.globl kerneltrap
.globl kernelvec
.align 4
kernelvec:
    # make room to save registers.
    addi sp, sp, -256
    # save caller-saved registers.
    sd ra, 0(sp)
    # .....
    sd t6, 240(sp)

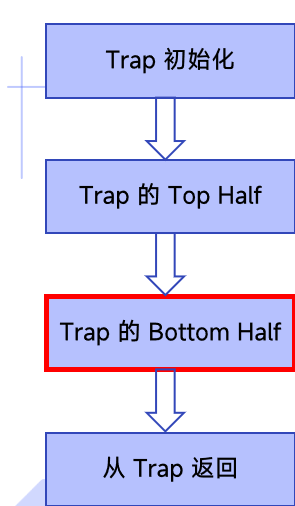
    # call the C trap handler in trap.c
    call kerneltrap

    # restore registers.
    ld ra, 0(sp)
    # .....
    ld t6, 240(sp)
    addi sp, sp, 256

    # return to whatever we were doing in the kernel.
    sret
```

调用 C 语言的 trap handler

Trap处理流程(S模式下)



```
.globl kerneltrap
.globl kernelvec
.align 4
kernelvec:
    # make room to save registers.
    addi sp, sp, -256
    # save caller-saved registers.
    sd ra, 0(sp)
    # .....
    sd t6, 240(sp)

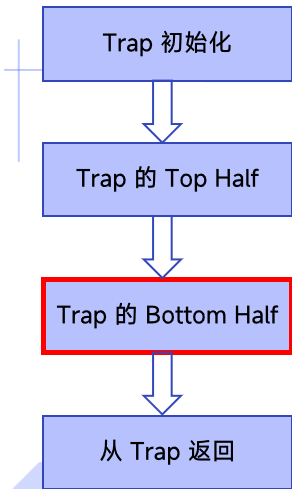
    # call the C trap handler in trap.c
    call kerneltrap

    # restore registers.
    ld ra, 0(sp)
    # .....
    ld t6, 240(sp)
    addi sp, sp, 256

    # return to whatever we were doing in the kernel.
    sret
```

恢复 (restore) 之前保存
的上下文的信息

Trap 处理流程(S模式下)



```
.globl kerneltrap
.globl kernelvec
.align 4
kernelvec:
    # make room to save registers.
    addi sp, sp, -256
    # save caller-saved registers.
    sd ra, 0(sp)
    # .....
    sd t6, 240(sp)

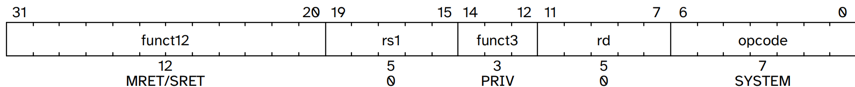
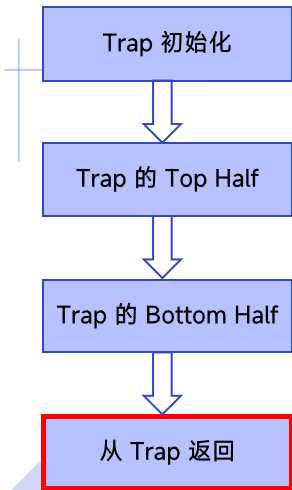
    # call the C trap handler in trap.c
    call kerneltrap

    # restore registers.
    ld ra, 0(sp)
    # .....
    ld t6, 240(sp)
    addi sp, sp, 256

    # return to whatever was in t0
    sret
```

执行 SRET 指令返回到
trap 之前的状态。

Trap 处理流程(S模式下)



- 针对不同权限级别下如何退出 trap 有各自的返回指令 xRET (x = M/S)
- 以在 S 模式下执行 sret 指令为例，会执行如下操作：
 - ✓ 返回后 Hart 的权限级别 = sstatus.SPP;
 - ✓ sstatus.SPP = 0;
 - ✓ sstatus.SIE = sstatus.SPIE;
 - ✓ sstatus.SPIE = 1
 - ✓ pc = sepc

Trap处理流程(S模式下)

```
void
kerneltrap()
{
    uint64 sstatus = r_sstatus();
    uint64 scause = r_scause();

    if((sstatus & SSTATUS_SPP) == 0)
        panic("kerneltrap: not from supervisor mode");
    if(intr_get() != 0)
        panic("kerneltrap: interrupts enabled");

    if (scause & 0x8000000000000000L) {
        printf("Asynchronous trap - interrupt!\n");
    } else {
        /* Synchronous trap - exception */
        printf("scause=0x%lx sepc=0x%lx stval=0x%lx\n", scause, r_sepc(), r_stval());
        panic("kerneltrap");
    }
}
```

Trap处理流程(S模式下)

```
void
kerneltrap()
{
    uint64 sstatus = r_sstatus();
    uint64 scause = r_scause();

    if((sstatus & SSTATUS_SPP) == 0)
        panic("kerneltrap: not from supervisor mode");
    if(intr_get() != 0)
        panic("kerneltrap: interrupts enabled");

    if (scause & 0x8000000000000000L) {
        printf("Asynchronous trap - interrupt!\n");
    } else {
        /* Synchronous trap - exception */
        printf("scause=0x%lx sepc=0x%lx stval=0x%lx\n", scause, r_sepc(), r_stval());
        panic("kerneltrap");
    }
}
```

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");

        /*
         * Synchronous exception code = 7
         * Store/AMO access fault
         */
        *(int *)0xFFFFFFFF = 100;

        task_delay(DELAY);
        yield();
    }
}
```

Trap处理流程(S模式下)

```
void
kerneltrap()
{
    uint64 sstatus = r_sstatus();
    uint64 scause = r_scause();

    if((sstatus & SSTATUS_SPP) == 0)
        panic("kerneltrap: not from supervisor mode");
    if(intr_get() != 0)
        panic("kerneltrap: interrupts enabled");

    if (scause & 0x8000000000000000L) {
        printf("Asynchronous trap - interrupt!\n");
    } else {
        /* Synchronous trap - exception */
        printf("scause=0x%lx sepc=0x%lx stval=0x%lx\n", scause, r_sepc(), r_stval());
        panic("kerneltrap");
    }
}
```

```
void user_task0(void)
{
    printf("Task 0: Created!\n");
    while (1) {
        printf("Task 0: Running...\n");

        /*
         * Synchronous exception code = 7
         * Store/AMO access fault
         */
        *(int *)0xFFFFFFFF = 100;

        task_delay(DELAY);
        yield();
    }
}
```

xv6 kernel is booting

Task 0: Created!

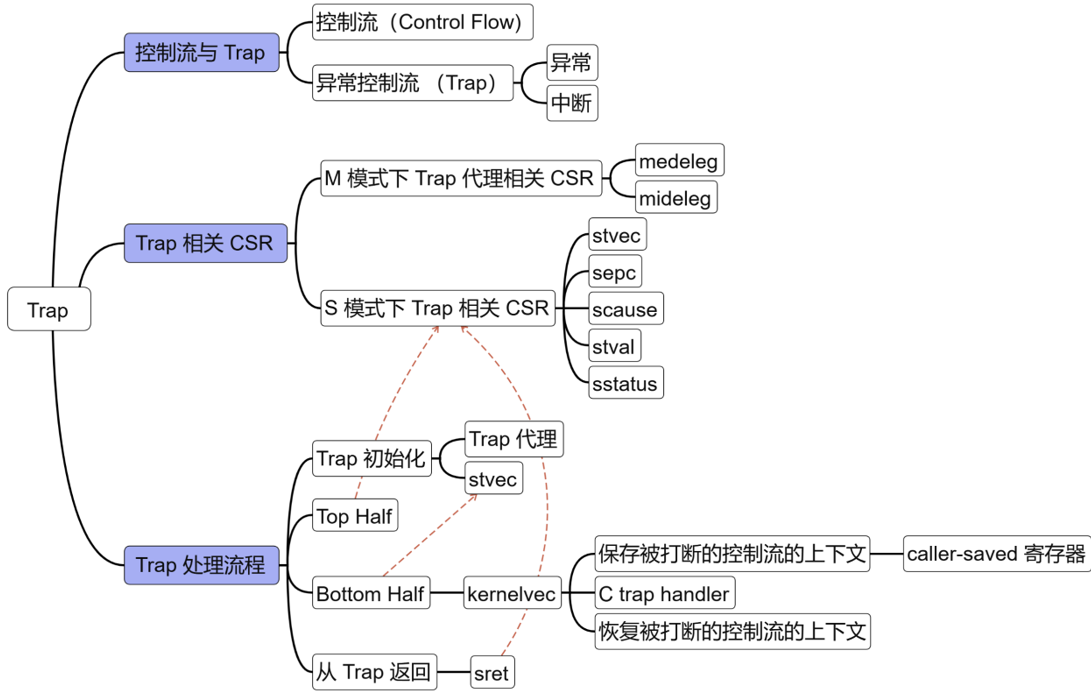
Task 0: Running...

scause=0x7 sepc=0x80000ab8 stval=0xffffffff

panic: kerneltrap



本章总结



谢谢

